

2026-7-28

使用 page_owner 定位内存 泄露

姓名 李昌全

掌锐电子

目录

一、page_ext 结构解析	2
1.1 page_ext	2
1.2 注册方式	3
1.3 内存布局	3
二、page_owner 结构解析	5
2.1 struct page_owner	5
2.2 启用与初始化	5
三、page_owner 定位内存泄漏原理	9
3.1 page_owner 判断内存泄露的逻辑	9
3.2 page_owner 定位内存泄露操作流程	9
四、page_owner 定位内存案例	10
4.1, page_owner 开启方式	10
4.2, 测试 demo	10
4.3, 测试数据分析	10

一、page_ext 结构解析

page_owner 依附在 page_ext 结构中，先分析 page_ext 结构。

1.1 page_ext

Linux 内核的 page_ext 机制是一种为每个物理页框 (page frame) 添加“扩展数据”的轻量级框架，它允许内核子系统在不修改核心 struct page 的情况下，为每个页框关联额外的私有数据 (例如 page owner 信息、页空闲跟踪等)。

1. struct page_ext_operations

```
// include/linux/page_ext.h
struct page_ext_operations {
    size_t offset; // 该模块的数据在 page_ext 结构中的偏移量 (由核心自动填充)
    size_t size; // 该模块需要的数据大小
    bool (*need)(void); // 判断是否需要实际分配内存
    void (*init)(void); // 分配完成后，模块的初始化回调 (可选)
};

struct page_ext {
    unsigned long flags;
};

static bool __init invoke_need_callbacks(void)
{
    int i;
    int entries = ARRAY_SIZE(page_ext_ops);
    bool need = false;

    for (i = 0; i < entries; i++) {
        if (page_ext_ops[i]->need && page_ext_ops[i]->need()) {
            page_ext_ops[i]->offset = page_ext_size;
            page_ext_size += page_ext_ops[i]->size;
            need = true;
        }
    }

    return need;
}
```

例如，page_owner 模块定义如下：

```
// mm/page_owner.c
static __init bool need_page_owner(void)
```

```
{  
    return page_owner_enabled;  
}  
  
static struct page_ext_operations page_owner_ops = {  
    .size = sizeof(struct page_owner), // 每个页额外存储的 page_owner 结构  
    .need = need_page_owner,  
    .init = init_page_owner,  
};
```

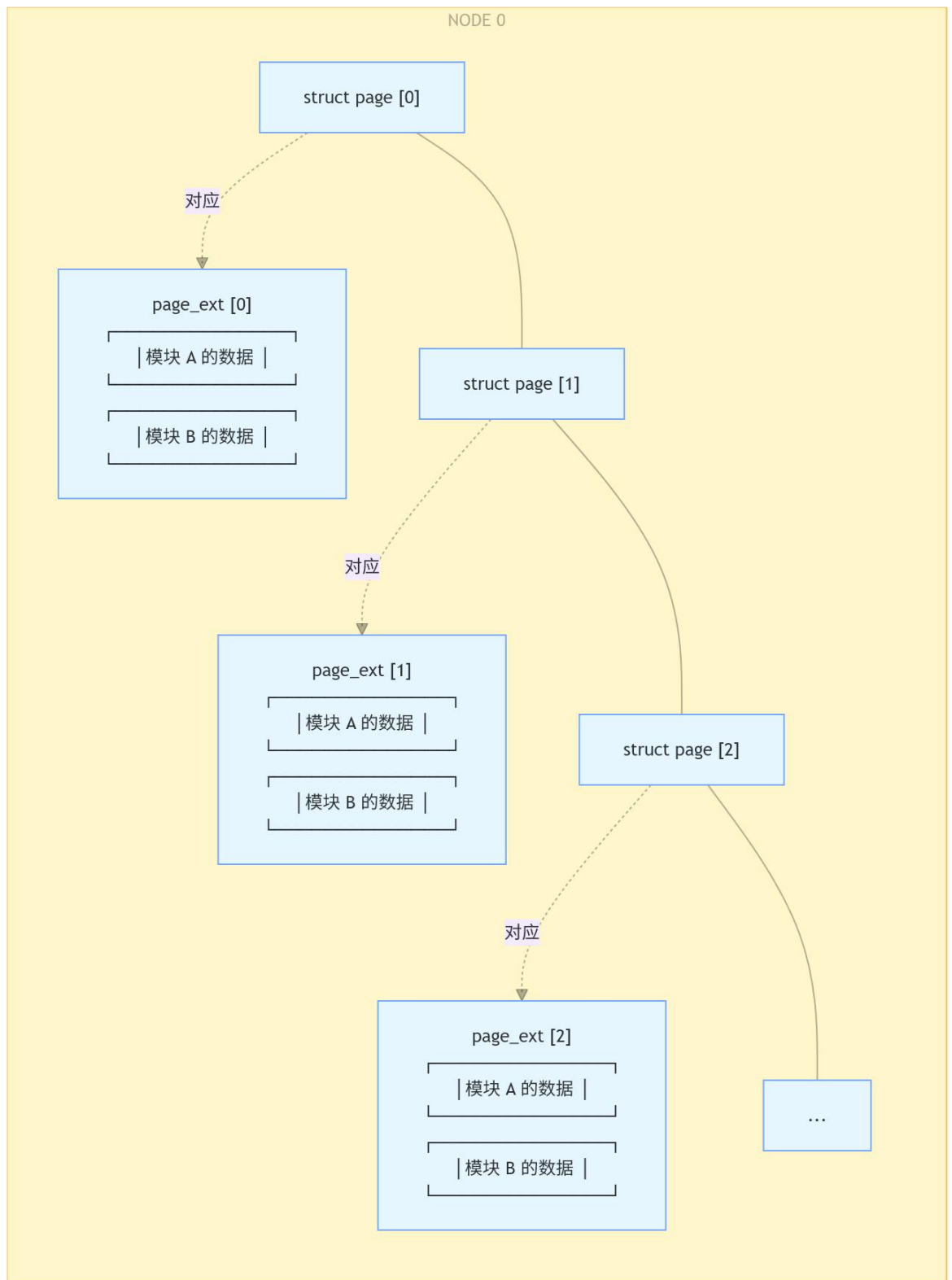
1.2 注册方式

所有这样的操作结构被统一收集到一个全局数组中：

```
// mm/page_ext.c  
static struct page_ext_operations *page_ext_ops[] = {  
#ifdef CONFIG_PAGE_OWNER  
    &page_owner_ops,  
#endif  
#ifdef CONFIG_PAGE_IDLE_FLAG  
    &page_idle_ops,  
#endif  
#ifdef CONFIG_PAGE_TABLE_CHECK  
    &page_table_check_ops,  
#endif  
    // ... 其他模块  
};
```

这个数组在编译时根据内核配置生成。

1.3 内存布局



关键点:

- 每个 `page_ext` 结构大小 = 所有激活模块的 `size` 之和（可能再加上一些对齐填充）。
- 模块的数据在 `page_ext` 中的位置由核心框架在运行时计算，保存在 `offset` 字段中。

二、page_owner 结构解析

2.1 struct page_owner

```
struct page_owner {
    unsigned short order; // 分配的阶 (order, 2^order 页)
    short last_migrate_reason;
    gfp_t gfp_mask; // 分配时使用的 GFP 标志
    depot_stack_handle_t handle; // 存储调用栈的句柄 (通过 stack_depot 保存)
    depot_stack_handle_t free_handle;
    u64 ts_nsec; // 申请时间
    u64 free_ts_nsec; // 释放时间
    char comm[TASK_COMM_LEN]; // 分配进程的 comm 名
    pid_t pid; // 分配进程的 PID
    pid_t tgid; // 线程 pid
};
```

`depot_stack_handle_t` 是一个句柄，指向由 **stack_depot** 存储的完整调用栈。`stack_depot` 是内核中一个高效的栈存储池，可以避免为每个页面重复保存相同的栈。

2.2 启用与初始化

1. 内核配置

需要开启 `CONFIG_PAGE_OWNER`，该选项会编译相关代码，并让 `page_ext` 预留 `page_owner` 的空间。

2. 启动参数

内核启动时必须传入 `page_owner=on`，否则 `page_owner` 处于禁用状态（不分配 `page_ext` 中的 `page_owner` 空间，也不做记录）。在 `mm/page_owner.c` 中：

```
static bool page_owner_enabled __read_mostly;
static int __init early_page_owner_param(char *buf)
{
    if (!buf)
        return -EINVAL;
    if (strcmp(buf, "on") == 0)
        page_owner_enabled = true;
    return 0;
}
```

```

}
early_param("page_owner", early_page_owner_param);

```

3. init 阶段会判断

init_page_owner 会先判断 page_owner_enabled:

```

static __init void init_page_owner(void)
{
    if (!page_owner_enabled)
        return;
    register_dummy_stack();
    register_failure_stack();
    register_early_stack();
    static_branch_enable(&page_owner_init);
    init_early_allocated_pages();
}

```

4. 分配时记录

当内核通过伙伴系统分配页面时（如 alloc_pages() 最终调用 post_alloc_hook()），会调用 set_page_owner() 来记录分配信息。调用路径大致为：

```

alloc_pages()
→ __alloc_pages()
→ get_page_from_freelist()
→ prep_new_page()
→ post_alloc_hook()
→ set_page_owner()

```

关键代码在 mm/page_owner.c:

```

void __set_page_owner(struct page *page, unsigned short order,
gfp_t gfp_mask)
{
    struct page_ext *page_ext;
    depot_stack_handle_t handle;
    handle = save_stack(gfp_mask); //保存堆栈信息
    page_ext = page_ext_get(page);
    if (unlikely(!page_ext))
        return;
    __set_page_owner_handle(page_ext, handle, order, gfp_mask);
    page_ext_put(page_ext);
}

```

```

static inline void __set_page_owner_handle(struct page_ext *page_ext,
depot_stack_handle_t handle,
unsigned short order, gfp_t gfp_mask)
{

```

```

    struct page_owner *page_owner;
    int i;
    for (i = 0; i < (1 << order); i++) {
        page_owner = get_page_owner(page_ext);

```

```

    page_owner->handle = handle;
    page_owner->order = order;
    page_owner->gfp_mask = gfp_mask;
    page_owner->last_migrate_reason = -1;
    page_owner->pid = current->pid;
    page_owner->tgid = current->tgid;
    page_owner->ts_nsec = local_clock(); //记录分配时间
    strncpy(page_owner->gcomm, current->group_leader->comm,
            sizeof(page_owner->gcomm));
    strncpy(page_owner->comm, current->comm,
            sizeof(page_owner->comm));
    __set_bit(PAGE_EXT_OWNER, &page_ext->flags);
    __set_bit(PAGE_EXT_OWNER_ALLOCATED, &page_ext->flags);
    memcpy(&page_owner->entry, &entry, sizeof(struct
            page_owner_callchain_entry));
    page_ext = page_ext_next(page_ext);
}
}

```

其中 `save_stack()` 会调用 `stack_depot_save()` 将当前的 `stack trace` 存入 `stack_depot`, 返回一个句柄。`stack_depot` 避免了为每个页面都保存一份完整的调用栈, 节省内存。

5, 释放时清空: `reset_page_owner`

当页面释放回伙伴系统时, 需要清除 `page_owner` 信息。调用路径为:

```

free_pages()
→ __free_pages()
    → free_pages_prepare()
        → free_pages_prepare()
            → kernel_poison_pages() 之前调用
                → reset_page_owner()

```

`mm/page_owner.c`:

```

void __reset_page_owner(struct page *page, unsigned short order)
{
    struct page_ext *page_ext = lookup_page_ext(page);
    int i;
    // 对于复合页, 需要清除所有子页的标志
    for (i = 0; i < (1 << order); i++) {
        struct page_owner *page_owner = get_page_owner(page_ext);
        // 清除标志, 并将句柄置 0
        __clear_bit(PAGE_EXT_OWNER, &page_ext->flags);
        page_owner->handle = 0;
        page_ext = page_ext_next(page_ext);
    }
}

```

释放时只是清除标志和句柄，并不删除 `stack_depot` 中的栈记录（因为其他页面可能仍在
使用同一栈记录，栈记录由 `stack_depot` 自动引用计数管理）。

6, 调试接口: `/sys/kernel/debug/page_owner`

`page_owner` 通过 `debugfs` 提供导出接口。在 `mm/page_owner.c` 中，定义了 `debugfs` 文件的
读回调函数：

```
static int __init page_owner_init(void)
{
    if (!page_owner_enabled)
        return 0;
    debugfs_create_file("page_owner", 0400, NULL, NULL, &page_owner_fops);
    return 0;
}
```

读回调函数 `page_owner_read()` 会遍历所有物理页面（或通过 `pfn` 迭代），对每个设置了
`PAGE_EXT_OWNER` 的页，将其记录信息格式化输出。

简化版的遍历逻辑：

```
for (pfn = 0; pfn < max_pfn; pfn++) {
    page = pfn_to_online_page(pfn);
    if (!page)
        continue;
    page_ext = lookup_page_ext(page);
    if (test_bit(PAGE_EXT_OWNER, &page_ext->flags)) {
        page_owner = get_page_owner(page_ext);
        // 输出分配信息: order, gfp_mask, pid, comm, 时间戳, 栈句柄对应的栈
        print_page_owner_info(buf, size, page_owner);
    }
}
```

当用户执行 `cat /sys/kernel/debug/page_owner` 时，就会得到所有当前已分配页面的记录。

每条记录包含：

- 分配阶 (order)
- GFP 标志 (可转换为可读字符串)
- PID 和进程名
- 时间戳
- 完整的调用栈 (通过 `stack_depot_fetch()` 从句柄恢复)

输出示例：

```
Page allocated via order 0, mask 0x12c40(GFP_NOFS|__GFP_NOWARN|__GFP_NORETRY), pid
902, tgid 902 (modprobe), ts 1418330541 ns, free_ts 1409087771 ns
```

```
PFN 268288 type Unmovable Block 262 type Unmovable Flags
0x4000000000000200(slab|zone=1|kasantag=0x0)
post_alloc_hook+0x1c0/0x1f4
prep_new_page+0x28/0x11c
get_page_from_freelist+0x154c/0x15b0
__alloc_pages+0xe0/0x238
alloc_slab_page+0xa4/0x1ac
new_slab+0xa4/0x350
__slab_alloc+0x534/0x8b8
__slab_alloc+0x6c/0xac
kmem_cache_alloc+0x1fc/0x358
...
```

三，page_owner 定位内存泄漏原理

3.1 page_owner 判断内存泄露的逻辑

- 1, 稳定复现泄露场景前后，抓取两次 page_owner 数据
 - 2, 过滤条件: is_free=0
 - 3, 对比两次快照: 新增大量同一条调用栈的 page → 该调用路径存在页泄露
 - 4, 观察 order、数量持续上涨，确认泄露
- 关键区分:

- 单次分配少量页面长期占用: 正常缓存 (pagecache、slab 页、buddy 预留)
- 持续不断新增同栈页面、不回落: 真正泄露

3.2 page_owner 定位内存泄露操作流程

- 1, 内核开启 CONFIG_PAGE_OWNER, CONFIG_DEBUG_FS
 - 2, cmdline 添加 page_owner=on 重启
 - 3, 挂载 debugfs: mount -t debugfs none /sys/kernel/debug
 - 4, 初始快照: cat /sys/kernel/debug/page_owner_show > page_1.txt
 - 5, 执行复现步骤 (反复操作, 让泄露增长)
 - 6, 第二次快照: cat /sys/kernel/debug/page_owner_show > page_2.txt
 - 7, 脚本过滤 is_free=0, 对比新增调用栈, 统计同栈 page 数量、order
- 可参照 debug_LostRAM 总结方式:



debug
LostRAM.rar

四，page_owner 定位内存案例

4.1, page_owner 开启方式

1, 合入 patch

```
yocto/src/bsp/lk2/app/blxboot/
diff --git a/app/blxboot/blxboot.c b/app/blxboot/blxboot.c
index 281fe40..60bbf10 100644
--- a/app/blxboot/blxboot.c
+++ b/app/blxboot/blxboot.c
@@ -328,6 +328,7 @@
/* bring up only*/
kcmline_append("arm64.nomte");
kcmline_append("arm64.nopauth");
+ kcmline_append("page_owner=on slub_debug=OFPU panic_on_taint=20
      stack_depot_disable=off");
/* bring up only*/
/* ab update*/
```

2, 检验方式

```
ls /sys/kernel/debug/page_owner
存在这 page_owner 开启
```

4.2, 测试 demo

驱动内存泄露 code

```
static ssize_t mysem_store(struct device *dev,
    struct device_attribute *attr,
    const char *buf, size_t count)
{
    char *ptr;
    int size = 0;
    int ret = 0;
    int i = 0;

    ret = kstrtoint(&buf[1], 10, &size);
    ptr = kmalloc(size, GFP_KERNEL);
    printk("licq buf:%s, size:%d, ptr=%p\n", buf, size, ptr);
    if ('k' == buf[0]) {
        size = size + 1;
    } else if ('m' == buf[0]) {
    }
    for (i=0; i < size; i++) {
        ptr[i] = 'a';
    }

    return count;
}
```

4.3, 测试数据分析

1, 先 dump 初始 page_owner 数据

```
cat /sys/kernel/debug/page_owner > init.txt
```

2, 执行泄露程序

```
echo m4096000 > /sys/devices/virtual/mysem/mysem/mysem_test
```

3, 再 dump page_owner 数据

```
cat /sys/kernel/debug/page_owner > current.txt
```

4, 使用工具检测内存泄露点

```
python ../page_owner_analysis 3.0.py init.txt current.txt
```

5, 分析数据

1) 查看存在 200MB 内存泄露

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
3	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	__kalloc_large_node(...)	__kalloc(...)	mysem_store(...)	dev_attr_store(...)	sysfs_kf_write(...)	kernfs_fop_write_iter(...)	vfs_write(...)	ksys_write(...)	__arm64_sys_write(...)	invoke_syscall(...)	el0_svc_common(...)	do_el0_svc(...)		
4	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	2083	10195	7512											
5	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	5345	11756	6411											
6	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	2160	6025	3802											
7	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	28	1840	1812											
8	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	18857	19067	210											
9	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	232	432	200											
10	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	1591	1560	169											
11	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	200	270	70											
12	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	18	82	64											
13	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	48	112	64											
14	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	4	61	57											
15	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	6	60	54											
16	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	137	191	54											
17	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	135	186	51											
18	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	240	288	48											
19	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	19	65	46											
20	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	1179	1221	42											
21	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	52	92	40											
22	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	234	270	36											
23	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	442	477	35											
24	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	89	123	34											
25	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	16	48	32											
26	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	0	32	32											
27	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	66	95	29											
28	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	15	44	29											
29	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	17	43	26											
30	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	40	71	25											
31	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	344	368	24											
32	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	12	36	24											
33	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	40	63	23											
34	post_alloc_hook(...)	prep_new_page(...)	get_page_from_freelist(...)	__alloc_pages(...)	270	292	22											

2) 查看泄露具体堆栈信息

打开 current.txt

泄露具体堆栈信息如下:

```
Page allocated via order 10, mask 0x140cc0(GFP_USER|__GFP_COMP), pid 3761, tgid 3761 (sh), ts 397472771332 ns, free_ts 155991389602 ns
PFN 862208 type Unmovable Block 842 type Unmovable Flags 0x4e8000000010000(head|zone=1|kasantag=0x3a)
post_alloc_hook+0x278/0x27c
prep_new_page+0x34/0x488
get_page_from_freelist+0x4094/0x411c
__alloc_pages+0x1f8/0x4e8
__kalloc_large_node+0xd0/0x22c
__kmalloc+0x130/0x250
mysem_store+0x78/0x180 [mysem]
dev_attr_store+0x5c/0x80
sysfs_kf_write+0x154/0x1ac
kernfs_fop_write_iter+0x2fc/0x48c
vfs_write+0x628/0x820
ksys_write+0xe8/0x1b4
__arm64_sys_write+0x90/0xb8
invoke_syscall+0x88/0x274
el0_svc_common+0x154/0x280
do_el0_svc+0x58/0x11c
```